Um Modelo para Gerenciamento de Perfis Distribuídos Através de Inferência em Trilhas

André Wagner
Jorge Luis Victória Barbosa

1 Introdução

Atualmente, aplicações têm se concentrado na coleta de informações a respeito dos usuários, com o objetivo de permitir que as aplicações possam conhecer suas preferências, possibilitando a oferta de serviços personalizados. Assim, estas aplicações organizam seu conhecimento a respeito do usuário em *perfis de usuário*, onde cada aplicação salva as informações em um formato baseado em um *modelo de usuário* específico.[1]

Sistemas sensíveis a contexto são aplicações nas quais o contexto atual de um usuário é levado em consideração na disponibilização de serviços. Dey[2] define contexto como qualquer informação que pode ser utilizada para caracterizar a situação de uma entidade, onde uma entidade é uma pessoa, local ou objeto considerado relevante à interação entre o usuário e a aplicação. Desta forma, o *perfil* torna-se parte essencial na descoberta de informações pertinentes aos usuários que possam ser utilizadas na construção de um contexto.

A capacidade de uma aplicação compreender o contexto de uma entidade pode ser amplificada através do uso de *trilhas*[3]. Uma trilha é o histórico de contextos visitados por uma entidade, e das ações realizadas pela entidade dentro do contexto.

A coleta de informações de um usuário pode ser *explícita* ou *implícita*[1]. Na coleta *explícita*, é requisitado ao usuário que preencha informações através de um formulário pré-definido, enquanto na coleta *implícita*, a informação é inferida a partir do estudo do comportamento dos usuários.

Informações a respeito de um usuário colhidas por uma aplicação podem ser úteis para outras aplicações que tenham o mesmo usuário [1]. No entanto, os diferentes *modelos de usuário* utilizados pelas aplicações tornam difícil a utilização de informações de um *perfil de usuário* de uma aplicação por outra. Além disso, algumas das informações do usuário podem ser válidas apenas dentro de um *contexto* específico, o que dificulta ainda mais que diferentes aplicações compartilharem o mesmo perfil.

Como forma de ultrapassar estas barreiras, o conceito de *personalização inter-sistemas* foi introduzido [4] como forma de compartilhar informações de usuários entre diferentes aplicações. Como o maior problema deste tipo de modelo é o alinhamento entre as diferentes abordagens de personalização, o estudo de *sistemas de modelos de usuário* é o principal desafio da pesquisa atual na área de personalização.[1]

O IEEE Standard Computer Dictionary [5] define interoperabilidade como "a capacidade de dois ou mais sistemas ou componentes trocarem informações e utilizar a informação que foi trocada." No âmbito da personalização, interoperabilidade é a "capacidade de acessar e interpretar informação derivada de múltiplas fontes heterogênias e integrar esta informação em um modelo de usuário de granularidade própria" [6]. Os dois principais modelos de interoperabilidade são: (i) *interoperabilidade sintática*, onde as diferenças entre os modelos de usuário são resolvidos em nível de aplicação, e (ii) *interoperabilidade semântica*, onde as diferenças entre os modelos são resolvidos em nível de conhecimento.

As duas principais abordagens utilizadas em sistemas de modelos de usuários interoperáveis são [1]:

Baseados em padrões : são baseados numa visão *top-down*, definindo um padrão que todas as aplicações envolvidas devem observar;

Baseados em mediação : se baseiam numa visão *bottom-up*, onde ocorre algum tipo de *reconciliação* entre os diferentes modelos de representação.

2 Trabalhos relacionados

Esta seção apresenta projetos relacionados a este, bem como uma comparação entre os mesmos.

2.1 PeLeP

O projeto PeLeP, apresentado por Levis et al[7], busca integrar a utilização de perfis com aperfeiçoamento automático com ambientes de educação ubíqua.

Os ambientes de *computação ubíqua* são parte de um cenário na qual serviços e informações estão disponíveis a qualquer momento, em qualquer lugar[8]. Esta visão vem sendo potencializada pela diminuição do custo e aumento da capacidade de memória e processamento dos dispositivos computacionais, bem como pela cada vez maior disponibilidade de recursos de localização e conectividade. A computação ubíqua enfatiza também a "tecnologia calma", onde a sobrecarga de informação é substituída pela disponibilização de recursos de forma inteligente, tomando a atenção do usuário apenas nos momentos em que é mais relevante.[9]

Levis et al compreendem *educação ubíqua* como sendo processos educacionais que ocorrem em qualquer lugar, a qualquer tempo e com qualquer dispositivo, de forma contínua, contextualizada e integrada ao cotidiano do aprendiz. Desta forma, o conhecimento presente no dia-a-dia das mais diferentes formas e em diferentes locais podem ser relacionados com processos educacionais direcionados aos aprendizes.

O PeLeP, cujo modelo é ilustrado na Figura 1, é um modelo dedicado ao gerenciamento de perfis de aprendizes em um ambiente ubíquo de ensino e aprendizagem. O modelo administra os perfis automaticamente, inferindo informações através do histórico do aprendiz. As alterações realizadas no perfil refletem o comportamento do aprendiz no ambiente ubíquo, levando em consideração o seu modelo de mobilidade e de contexto.

O *perfil do aprendiz* consiste em um perfil contendo campos pré-definidos, voltados para o ambiente de educação ubíqua. Os campos são distribuídos nas seguintes categorias: identificação, objetivos, agenda, segurança, relacionamentos, competências e preferências.

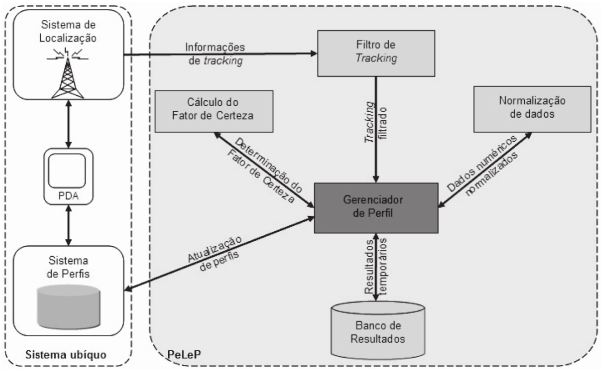

Figura 1: Arquitetura do PeLeP[7]

É realizado um acompanhamento (*tracking*) do histórico do aprendiz, onde são armazenados os contextos visitados pelo usuário, bem como os conteúdos, dispositivos e aplicativos utilizados em cada contexto. Os valores relativos das variáveis numéricas recebidas no *tracking* são regularizados através de um processo de normalização de amplitude. Como diferentes valores podem ter diferentes impactos para a determinação da preferência de um usuário, calcula-se o Fator de Certeza (FC), de acordo com a equação 1, onde f_i é o valor normalizado para o fator, p_i o seu percentual, e n o número de fatores considerados.

$$FC = \frac{\frac{\sum_{i=0}^{n} f_i p_i}{100}}{n} \tag{1}$$

O fator tempo também é levado em consideração, onde Fatores de Certeza mais antigos tem um impacto menor. Assim, por exemplo, se o usuário trocar o seu aplicativo preferido, à medida que o tempo for passando o impacto do aplicativo anterior será cada vez menor.

O processo de atualização do modelo do aprendiz é então realizado através de regras pré-definidas, que levam em conta as ocorrências e o seu fator de certeza. As regras suportam conclusões a partir de evidências antecedentes, e cada categoria de informação possui regras diferentes.

2.2 Chellouche et al

O artigo de Chellouche et al[10] discute as dificuldades atuais no desenvolvimento de aplicativos para dispositivos móveis, uma vez que os mesmos possuem uma grande variedade de resolução, memória, poder computacional, etc. Além disso, a qualidade da conexão de um usuário pode variar grandemente, e isto também deve ser levado em consideração no desenvolvimento.

O conjunto de informações que caracteriza um usuário é chamada de *perfil de usuário*. Com a intenção de compartilhar o contexto do usuário entre serviços e minimizar as mudanças no desenvolvimento de aplicações sensíveis a contexto, os autores propõem como solução um *middleware* que gerencia o perfil de um usuário dinamicamente e supre às aplicações todas as informações que elas podem precisar durante o período de adaptação.

O perfil do usuário é constituído por seu contexto, onde a definição de contexto utilizada é aquela dada por Dey[2], na qual contexto refere-se a qualquer informação que pode ser utilizada para caracterizar a situação de uma entidade. Assim, o perfil de usuário é dividido em cinco componentes:

Perfil geral : refere-se às informações básicas, como nome, endereço, telefone, idade, sexo, etc. Inclui também possíveis deficiências pessoas do usuário.

Perfil do dispositivo : informações a respeito dos dispositivos do usuário, tais como marca, capacidade, sistema operacional, etc.

Perfil de rede : descreve as redes às quais o usuário pode se conectar, com suas características.

Perfil dos serviços : registra as informações a respeito de serviços, como nome, versão, porta e conteúdo.

Perfil de contexto : contém informações a respeito do ambiente do usuário, como hora, data e localização, aplicações em execução, banda disponível, etc. É considerada a parte dinâmica do perfil, composta por dados voláteis.

O *middleware* é apresentado segundo ilustrado na Figura 2. O *middleware* é composto dos seguintes módulos:

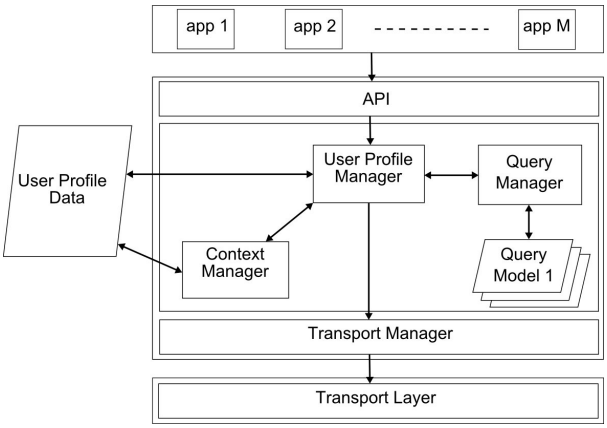

Figura 2: Arquitetura do *middleware* de Chellouche et al[10]

User profile manager : módulo central da arquitetura, que interage com todos os outros módulos com o objetivo de construir o perfil correto.

Context manager : monitora o ambiente, coletando informações com o objetivo de atualizar a parte dinâmica do perfil.

Query manager : módulo responsável por validar e salvar um modelo de consulta para cada aplicação. É construída quando a aplicação é instalada e utilizada quando a aplicação solicita informações.

Transport manager : faz a comunicação do perfil de usuário para a aplicação.

API : constitui o ponto de interface entre as aplicações e o middleware.

O perfil é construído de acordo com a Figura 3. A aplicação solicita ao gerenciador de perfis, através da API, que por sua vez repassa a solicitação ao gerenciador de consultas. Simultâneo a isto, o gerenciador de contextos é informado que a aplicação está em execução e passa a atualizar o gerenciador de perfis com as informações atualizadas. O perfil de usuário é criado e enviado ao gerenciador de transporte, que retorna o perfil solicitado à aplicação solicitante.

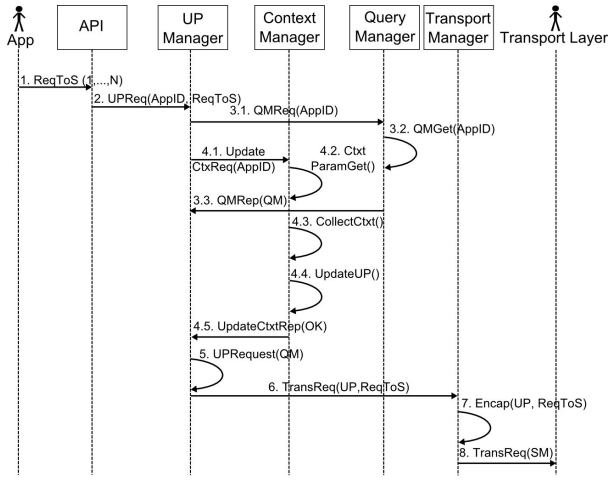

Figura 3: Construção de perfil[10]

2.3 UUCM

O artigo apresentado por Niederée et al[4] busca responder três indagações, que são apresentadas como sendo os três desafios básicos da personalização:

- Como modelar o usuário e sua situação de modo extensível e unificado, de forma que este modelo possa ser interpretado e usado para personalização por diferentes sistemas?
- Como coletar dados para preencher e atualizar perfis de usuário a partir das interações do usuário com diferentes sistemas baseados em um modelo de contexto de usuário unificado?
- Como habilitar conceitualmente e tecnicamente diferentes sistemas a fazer uso dos perfis de usuário coltados para aperfeiçoamento do suporte ao usuário?

Os autores apresentam como resposta a estas indagações um modelo denominado UUCM, que tem como objetivo modelar características do usuário e de seu contexto. O conceito de contexto utilizado é aquele apresentado por Berenetti et al[11], na qual o contexto pode ser pensado como as informações "extra", geralmente implícitas (como associações, fatos, suposições), que podem ser ser utilizadas para compreender plenamente uma interação, comunicação ou representação de conhecimento.

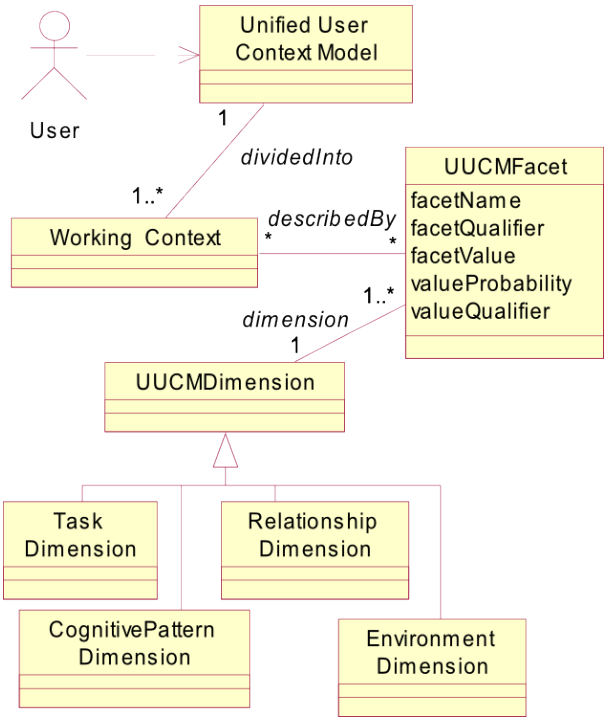

Figura 4: Modelo do UUCM[4]

A abordagem para a modelagem unificada de contextos de usuário é proposta em dois níveis. No nível *abstrato*, são definidos os blocos básicos de construção: contexto de usuário, facetas do modelo de usuário, propriedades principais e dimensões do modelo de usuário. O termo *faceta* representa as diferentes características do usuário. Este nível define um metamodelo para as dimensões e facetas concretas utilizadas na descrição do modelo de contexto de usuário. Para que a abordagem de personalização multi-sistema seja válida, é necessário que que este metamodelo de contexto de usuário seja publicado como uma ontologia compartilhada, e que todos os participantes utilizem este modelo.

No nível *concreto*, um conjunto de dimensões e facetas são definidos. As facetas e dimensões são descritas como parte de uma ontologia adicional que é compartilhada pelos componentes envolvidos na personalização multi-sistemas.

O modelo do UUCM é apresentado na Figura 4. Segundo o modelo, um perfil de usuário (*Unified User Context Model*) é dividido em um conjunto de contextos de trabalho (*Working Contexts*), onde cada contexto é descrito por um con-

junto de facetas (*UUCM facet*). Cada faceta é vinculada a uma dimensão (*UUCMDimension*).

O UUCM apenas define o método através da qual o perfil de contexto de usuário é descrito e estruturado para personalização multi-sistema. Para a descrição de perfis de contexto concretos, o UUCM utiliza ontologias (ou vocabulários) para as dimensões, facetas e valores de faceta.

As seguintes dimensões são utilizadas:

Cognitiva (*CognitivePattern*): descreve as dimensões cognitivas do usuário, tais como *áreas de interesse*, *competências* e *preferências*;

Tarefa (*Task*): descreve os objetivos do usuário quando ele interage com sistemas. Utiliza dados tais como *tarefa atual*, *papel do usuário na tarefa* e *histórico da tarefa*;

Relacionamentos (*Relationship*): armazena as entidades com as quais o usuário tem alguma relação;

Ambiente (*Environment*): parâmetros que são tipicamente utilizados para abordagens sensíveis a contexto, tais como *tempo*, *localização*, *dispositivo* e *linguagem*.

O perfil de usuário do UUCM é codificado como um *RDF Schema*[12], ampliado com expressões *OWL*.

O modelo inclui ainda o conceito de *passaporte de contexto*. O passaporte de contexto é uma representação compacta do modelo de contexto atual do usuário para a personalização multi-sistemas. Ele contém informações ontologicamente arranjadas a respeito dos padrões cognitivos (habilidades, interesses, etc), o ambiente (data, local, dispositivo usado) e sua rede pessoal de pessoas e relacionamentos envolvidos.

Para utilizar o passaporte de contexto em um ambiente de personalização multi-sistemas, o usuário "apresenta" o passaporte ao sistema que o usuário deseja utilizar. Como o passaporte está vinculado a uma ontologia compartilhada, é possível que o sistema consiga interpretar parcialmente o passaporte através de uma arquitetura mediadora. Como resultado, dois fluxos são possíveis: (i) do passaporte para o sistema, auxiliando o sistema a compreender o contexto do usuário, e (ii) do sistema para o passaporte, realizando alterações nas informações de contexto do usuário.

2.4 Avaliação dos Ambientes Pesquisados

O PeLeP[7] é um modelo que suporta o aperfeiçoamento automático do perfil do aprendiz em ambientes de educação ubíqua. Ele usa o histórico dos aprendizes nos contextos de um ambiente ubíquo para atualização dos perfis. O perfil do aprendiz no modelo é refinado e enriquecido através de inferências. As inferências são baseadas na mobilidade do aprendiz e no monitoramento das tarefas que ele executa nos contextos de um ambiente de educação ubíqua.

O modelo apresentado por Chellouche et al[10] apresenta um *middleware* que pode ser usado por diferentes aplicativos como um servidor central de perfis. Ele é voltado para aplicações móveis e, embora atualize parte do perfil como forma de oferecer suporte a sensibilidade a contexto, não permite que os aplicativos criem novos campos nem definam as regras para atualização dos campos existentes.

O UUCM[4] é um modelo que pode ser utilizado para modelar as características de um usuário e seu contexto em ambientes de personalização de múltiplos sistemas. A proposta diz respeito a uma ontologia básica, baseada em contexto, a qual os sistemas podem estender, desde que os sistemas que usem este modelo concordem em uma ontologia básica. Desta forma, o modelo busca ser uma resposta a um dos principais desafios da personalização: como modelar o perfil de usuário e seu contexto de forma extensível e unificada, de modo que possa ser interpretada e utilizada por diferentes sistemas.

A Tabela 1 exibe um comparativo dos trabalhos apresentados, onde os seguintes fatores são levados em consideração:

Interoperabilidade: o modelo apresenta interoperabilidade – caso positivo, se a mesma é sintática ou semântica;

Perfis dinâmicos: o modelo de perfil permite que os perfis sejam atualizados de acordo com eventos externos, sem a interferência do usuário;

Armazenamento: define onde ficam armazenados os perfis do usuário;

Sensível a contexto: o contexto do usuário é levado em consideração na formação do perfil;

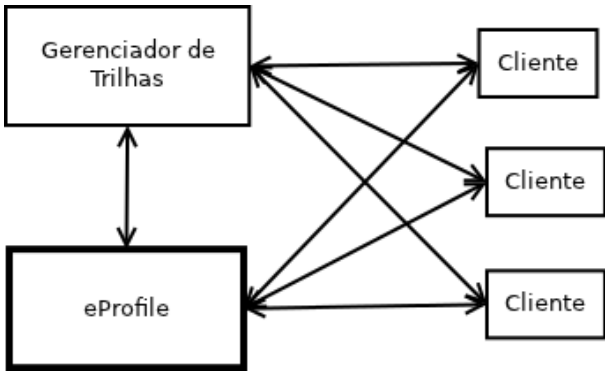

Figura 5: Modelo do eProfile

Utiliza trilhas: as trilhas do usuário são levadas em consideração na construção do perfil;

Domínio: o perfil é voltado para um domínio específico, ou se o mesmo permite que vários domínios possam ser utilizados.

3 eProfile

O eProfile é um gerenciador de perfis voltado para ambientes distribuídos, que infere dados de perfil através da análise das trilhas de uma entidade (disponibilizadas por um gerenciador de trilhas), seguindo regras definidas pelas próprias entidades, e disponibilizado os dados inferidos para serem utilizados por múltiplas aplicações.

A Figura 5 apresenta o modelo, que é organizado em três partes.

O eProfile é o modelo apresentado neste artigo. Ele interage com o gerenciador de trilhas através de uma interface padronizada, de onde busca as informações necessárias para a inferência dos perfis.

Para que o eProfile possa operar, é necessário que o mesmo esteja conectado a um gerenciador de trilhas que implemente uma interface padrão. O gerenciador de trilhas é o módulo responsável por armazenar as trilhas, fornecendo uma interface que permita a consulta de um conjunto destas trilhas dentro de parâmetros específicos (por exemplo, entidade ou data). O gerenciador de trilhas deve fornecer as trilhas no formato especificado pelo eProfile.

Os clientes referem-se a dispositivos que se comunicam com o gerenciador de trilhas e com o eProfile. Estes clientes fazem o registro de seus eventos dentro do gerenciador de trilhas. O eProfile e o gerenciador de trilhas trabalham em conjunto, independente dos clientes, de modo a manter as informações de perfil preprocessadas dentro do eProfile.

Cada cliente é responsável por fornecer o conjunto de regras para o processamento de suas próprias trilhas. Desta forma, cada cliente pode registrar os eventos de histórico no formato que considerar pertinente, desde que as regras respeitem o formato utilizado.

4 Modelo de Inferência

Para o armazenamento e processamento das ontologias neste trabalho, foram estudadas algumas linguagens de representação de conhecimento.

A linguagem **CycL** é a linguagem utilizada pelo o projeto de *Cyc*[13]. O Cyc é um projeto de inteligência artificial que tem o objetivo de montar uma ontologia e base de conhecimento a partir do senso comum, com o propósito de permitir que aplicações de inteligência artificial realizem raciocínio semelhante ao humano. A linguagem CycL é uma linguagem declarativa baseada em lógica de primeira ordem clássica, com extensões para operadores modais e lógica de ordem superior. A base de conhecimento é dividida em micro-teorias (Mt), que são coleções de conceitos e fatos tipicamente pertencentes a um domínio particular de conhecimentos e, ao contrário da base de conhecimento como um todo, cada micro-teoria deve estar livre de contradições.

RDF (*Resource Description Framework*) é uma linguagem utilizada para representação de metadados a respeito de recursos na Web[12]. No entanto, o conceito de "recurso da Web" pode ser generalizado, de forma que o RDF pode ser utilizado para representar informações a respeito de coisas que podem ser identificadas na Web, mesmo que elas não possam ser carregadas da Web. Recursos podem ser recursos eletrônicos, como arquivos, ou conceitos, como pessoas. O RDF é complementado pelo **RDF/S** (*RDF/Schema*), uma extensão semântica do RDF. O RDF/S é uma linguagem de descrição de vocabulário que provê mecanismos para descrever grupos de recursos relacionados e os relacionamentos entre estes recursos[14].

O **DAML+OIL** é uma linguagem de representação de conhecimento baseada nas linguagens

Trabalho	Interoperabilidade	Perfis dinâmicos	Armazenamento	Sensibilidade a contexto	Utiliza trilhas	Domínio
PeLeP [7]	Não	Sim	Servidor	Sim	Parcial	Educação
Chellouche et al [10]	Sintática	Sim	Servidor	Sim	Não	Serviços multimídia
UUCM [4]	Sintática	Sim	Cliente	Sim	Não	Genérico

Tabela 1: Comparação entre os trabalhos apresentados

DAML e OIL. O DAML (*DARPA Agent Markup Language*) foi desenvolvido pelo Centro de Pesquisas Avançadas da Defesa Americana e é uma linguagem de marcação de agentes baseada no RDF. Já o OIL (*Ontology Inference Layer*) é uma linguagem com os mesmos objetivos, criada pelo por um grupo de universidades europeias. Ambos os esforços foram unidos na criação do DAML+OIL, que por sua vez deu origem ao OWL[15].

O **OWL** é a linguagem de ontologia mais expressiva definida ou em definição para a Web Semântica[15]. O OWL utiliza URI[1] para os nomes, e a estrutura de descrição disponibilizada pelo RDF. O OWL está uma camada acima do RDF e do RDF/S, e adiciona mais vocabulário para descrever propriedades e classes: entre outros, relações entre classes, cardinalidade, igualdade, tipagem mais avançada de propriedades, características de propriedades e classes enumeradas[16]. O OWL está disponível em três sublinguages: *OWL Lite*, *OWL DL* e *OWL Full*, cada uma aumentando o nível de complexidade e expressividade. Um grande número de ferramentas com capacidade para edição e raciocínio semântico com suporte para OWL está disponível no mercado.

Devido à expressividade e ao suporte de ferramentas externas, o OWL será a linguagem de ontologias utilizado pelo eProfile para modelagem da ontologia do perfil.

Existem diversos raciocinadores que oferecem suporte a OWL, entre os quais estão:

Pellet: é um racionador com suporte a OWL DL, que oferece bom desempenho e um *middleware* extensível. O Pellet é escrito em Java pode ser utilizado para raciocinar com tipos de dados definidos pelo usuário e provê suporte para depuração em ontologias[17].

FaCT++: é um raciocinador de lógica proposicional criado como uma plataforma para experimentar com novos algoritmos de *tableaux* semântico e técnicas de otimização. Ele incorpora a maior parte das técnicas de otimização padrão, mas inclui algumas novas[18].

HermiT: é o primeiro raciocinador OWL que se baseia em um cálculo *hypertableaux*, que provê desempenho superior a qualquer algoritmo anterior[19]. O HermiT é disponibilizado como um plugin do Protegé, como uma ferramenta de linha de comando ou como uma biblioteca Java.

[1] *Uniform Resource Identifier*

Referências

[1] M. Viviani, N. Bennani, and E. Egyed-Zsigmond, "A Survey on User Modeling in Multi-Application Environments," in *The Third International Conference on Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services CENTRIC'10*, pp. 111–116, IEEE, Aug. 2010.

[2] A. Dey, "Understanding and using context," *Personal and Ubiquitous Computing*, vol. 6, no. 1, pp. 4–7, 2001.

[3] J. Silva, J. Rosa, J. Barbosa, D. N. F. Barbosa, and L. A. M. Palazzo, "Content distribution in trial-aware environments," in *Proceedings of the XV Brazilian Symposium on Multimedia and the Web*, WebMedia '09, (New York, NY, USA), pp. 15:1–15:8, ACM, 2009.

[4] C. Niederée, A. Stewart, B. Mehta, and M. Hemmje, "A Multi-Dimensional, Unified User Model for Cross-System Personalization," in *E4PIA Workshop 2004*, 2004.

[5] IEEE, "IEEE standard computer dictionary. a compilation of IEEE standard computer glossaries," *IEEE Std 610*, p. 1, 1991.

[6] F. Carmagnola and F. Cena, "User identification for cross-system personalisation," *Inf. Sci.*, vol. 179, pp. 16–32, January 2009.

[7] D. Levis, J. L. V. Barbosa, and S. C. C. S. Pinto, "Aperfeiçoamento automático do perfil do aprendiz em ambientes de educação ubíqua," *Revista Brasileira de Informática na Educação*, vol. 16, no. 1, 2008.

[8] M. Weiser, "The computer for the 21st century," *Scientific America*, pp. 94–104, 1991.

[9] M. Weiser and J. S. Brown, "Designing calm technology," *PowerGrid Journal*, July 1996.

[10] S. A. Chellouche, J. Arnaud, and D. Négru, "Flexible user profile management for context-aware ubiquitous environments," in *Proceedings of the 7th IEEE conference on Consumer communications and networking conference*, CCNC'10, (Piscataway, NJ, USA), pp. 980–984, IEEE Press, 2010.

[11] M. Benerecetti, P. Bouquet, and C. Ghidini, "Contextual reasoning distilled," *Philosophical Foundations of Artificial Intelligence. A special issue of the journal of Experimental and Theoretical AI (JETAI)*, vol. 12, no. 3, pp. 279–305, 2000.

[12] W3C, "RDF/XML syntax specification (revised)," 2011. `http://www.w3.org/RDF/`.

[13] D. B. Lenat and R. V. Guha, *Building Large Knowledge-Based Systems; Representation and Inference in the Cyc Project.* Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1st ed., 1989.

[14] W3C, "RDF vocabulary description language 1.0: Rdf schema," 2011. `http://www.w3.org/TR/rdf-schema/`.

[15] M. C. Daconta, L. J. Obrst, and K. T. Smith, *The Semantic Web : A Guide to the Future of XML, Web Services, and Knowledge Management.* Indianapolis: Wiley, May 2003.

[16] W3C, "OWL web ontology language guide," 2011. `http://www.w3.org/TR/owl-guide/`.

[17] E. Sirin, B. Parsia, B. Grau, A. Kalyanpur, and Y. Katz, "Pellet: A practical OWL-DL reasoner," *Web Semantics: Science, Services and Agents on the World Wide Web*, vol. 5, pp. 51–53, June 2007.

[18] D. Tsarkov and I. Horrocks, "Fact++ description logic reasoner: System description," in *Proc. of the Int. Joint Conf. on Automated Reasoning (IJCAR 2006)*, vol. 4130 of *Lecture Notes in Artificial Intelligence*, pp. 292–297, Springer, 2006.

[19] B. Motik, R. Shearer, and I. Horrocks, "Hypertableau Reasoning for Description Logics," *Journal of Artificial Intelligence Research*, vol. 36, pp. 165–228, 2009.